

Perfect ✅ — let's rebuild **Day 1** using **YOUR** current template + final architecture direction (pymupdf + pdfplumber ready)

👉 This is **clean, production-compatible Day 1** (no rework later)

🚀 ✅ DAY 1 — PROJECT FOUNDATION (FINAL VERSION)

✅ 🎯 GOAL (Day 1)

- ✓ Setup repo
 - ✓ Setup folder structure (correct for future)
 - ✓ Setup config (free/paid toggle)
 - ✓ Setup API base
 - ✓ Prepare for Day 2/3 (parser ready)
-

✅ 📁 FINAL FOLDER STRUCTURE (CREATE EXACT)

project_root/

|

|— app/

| |— config.py ✅

|

| |— api/

| | |— main.py ✅

|

| |— ingestion/

| | |— pipeline.py ✅

|

| |— parser/

| | |— parser_router.py ✅

| | |— table_router.py ✅

```
| | ├── pymupdf_parser.py ❌ (Day 2)
| | ├── pdfplumber_table.py ❌ (Day 3)
| | ├── gcp_parser.py ✅ stub
| | ├── gcp_table_parser.py ✅ stub
|
| ├── schema/
| | ├── models.py ✅
| | ├── mapper.py ✅
|
| ├── storage/
| | └── db.py ✅
|
| ├── query/ ❌ (later)
| ├── agent/ ❌ (later)
| ├── models/ ❌ (later)
|
| ├── tests/
| | └── test_ingestion.py ✅
|
| ├── data/
|
| ├── requirements.txt ✅
| └── .env
```

✅✅ STEP 1 — ENV SETUP

```
python -m venv venv
```

```
source venv/bin/activate # or Windows
```

```
pip install fastapi uvicorn pymupdf pdfplumber duckdb pandas
```

✅✅ STEP 2 — CONFIG (VERY IMPORTANT)

📄 app/config.py

```
MODE = "free" # change to "paid" later
```

✅✅ STEP 3 — API BASE

📄 app/api/main.py

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def health():
```

```
    return {"status": "ok"}
```

✅ RUN

```
uvicorn app.api.main:app --reload
```

✅✅ STEP 4 — PARSER ROUTER (FUTURE READY)

📄 app/parser/parser_router.py

```
from app.config import MODE
```

```
def parse_document(file_path):
```

```
    if MODE == "paid":
```

```
        from app.parser.gcp_parser import parse_gcp
```

```
        return parse_gcp(file_path)
```

```
# Day 2 will plug pymupdf here
```

```
return {
```

```
"titles": [],  
"text": []  
}
```

✅✅ STEP 5 — TABLE ROUTER (FUTURE READY)

 **app/parser/table_router.py**

```
from app.config import MODE
```

```
def get_tables(file_path, parsed):
```

```
    if MODE == "paid":
```

```
        from app.parser.gcp_table_parser import gcp_table_extract
```

```
        return gcp_table_extract(file_path)
```

```
# Day 3 will plug pdfplumber here
```

```
return []
```

✅✅ STEP 6 — PIPELINE (CORE FLOW)

 **app/ingestion/pipeline.py**

```
from app.parser.parser_router import parse_document
```

```
from app.parser.table_router import get_tables
```

```
def process_pdf(file_path):
```

```
    parsed = parse_document(file_path)
```

```
    tables = get_tables(file_path, parsed)
```

```
    return {
```

```
        "titles": parsed.get("titles", []),
```

```
"text": parsed.get("text", []),  
"tables": tables  
}
```

✅✅ STEP 7 — STORAGE BASE (DuckDB)

 **app/storage/db.py**

```
import duckdb  
  
conn = duckdb.connect("data.db")  
  
conn.execute("""  
CREATE TABLE IF NOT EXISTS metrics (  
    component TEXT,  
    value FLOAT,  
    condition TEXT  
)  
""")
```

✅✅ STEP 8 — SCHEMA (INITIAL ONLY)

 **app/schema/models.py**

```
def create_record(component, value, condition):  
    return {  
        "component": component,  
        "value": value,  
        "condition": condition  
    }
```

✅✅ STEP 9 — TEST FILE

tests/test_ingestion.py

```
from app.ingestion.pipeline import process_pdf
```

```
result = process_pdf("data/sample.pdf")
```

```
print("\n=== OUTPUT ===\n")
```

```
print(result)
```

STEP 10 — REQUIREMENTS FILE

requirements.txt

```
fastapi
```

```
uvicorn
```

```
pymupdf
```

```
pdfplumber
```

```
duckdb
```

```
pandas
```

DAY 1 OUTPUT (EXPECTED)

Run:

```
python -m tests.test_ingestion
```

 Output:

```
{  
  "titles": [],  
  "text": [],  
  "tables": []  
}
```

THIS IS CORRECT FOR DAY 1

✅✅ WHERE YOU ARE AFTER DAY 1

Infra ✅

Router ✅

Pipeline ✅

Storage ✅

Parser ❌ (next)

Tables ❌ (next)

Schema ❌ (next)

✅✅ WHAT COMES NEXT

🚀 **Day 2:**

pymupdf (text extraction)

🚀 **Day 3:**

pdfplumber (table extraction + cleaning)

✅✅ FINAL MESSAGE

👉 This Day 1 is:

✓ aligned with your architecture ✅

✓ ready for real parser ✅

✓ supports FREE + PAID ✅

✓ no rework later ✅

✅ Say **Day 2** 🚀